# 1. Title Page

**Lab Name:** Lab 06: TCP Connection, Flow, and Congestion Control **Date:** YYYY-MM-DD **Name:** [Your Name] **Lab Partners:** [Partners' Names, if any] **Software/Hardware Used:** Cisco Modeling Labs (CML) - Personal / Free Edition

---

# 2. Background

The Transmission Control Protocol (TCP) is a core Transport Layer protocol designed to provide reliable, ordered, and error-checked delivery of data between applications. To achieve this, TCP relies on:

1. **Connection Management**: A three-way handshake (SYN, SYN-ACK, ACK) to establish connection state, and a four-step teardown (FIN, ACK) to terminate it.
2. **Reliable Data Transfer**: Using Sequence (Seq) and Acknowledgment (Ack) numbers to track byte streams, acknowledge received data, and trigger retransmissions for lost segments.
3. **Flow Control**: Using a sliding window mechanism where the receiver advertises its available buffer space (Window Size) to prevent the sender from overflowing its queue.
4. **Congestion Control**: Adjusting the sending rate dynamically (slow start and congestion avoidance) based on perceived network capacity, packet loss, and round-trip time (RTT).

In this lab, you will generate high-volume TCP traffic across a constrained routing topology in CML, analyze the packets in Wireshark, and plot TCP congestion graphs.

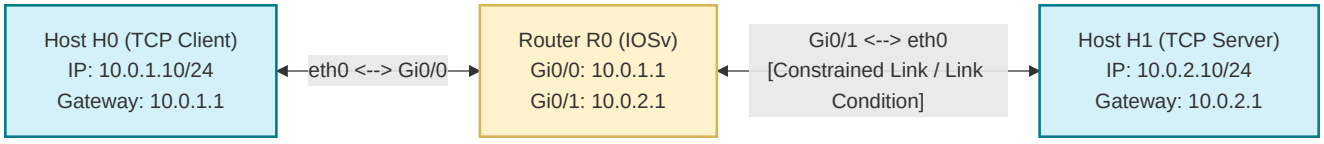
---

# 3. Objective / Purpose

To analyze connection establishment/termination, trace reliable data transfer (Seq/Ack tracking), examine flow control advertising (Window Size), calculate throughput, and graph TCP congestion control behavior under constrained link conditions.

---

# 4. Topology Diagram




**Details:**

- 2x Hosts ( `<initials>-H0` and `<initials>-H1` ) communicating over a routed CML topology via Router ( `<initials>-R0` ).
- Link constraints (bandwidth limits and latency) are applied to the link between R0 and H1 to force congestion control behavior.

---

# 5. Equipment & Environment

- **Software:** Cisco Modeling Labs (CML), Terminal Emulator (e.g., PuTTY, SecureCRT, native terminal), Wireshark
- **Hardware / Virtual Nodes:** 2x Linux Nodes (configured as hosts), 1x IOSv Router

---

# 6. Configuration / Methodology

### IP Addressing Table

| Device | CML Node Name | Interface | IP Address | Subnet Mask | Gateway |
|---|---|---|---|---|---|
| Client | `<initials>-H0` | eth0 | 10.0.1.10 | 255.255.255.0 | 10.0.1.1 |
| Server | `<initials>-H1` | eth0 | 10.0.2.10 | 255.255.255.0 | 10.0.2.1 |
| Router | `<initials>-R0` | G0/0 | 10.0.1.1 | 255.255.255.0 | N/A |
| Router | `<initials>-R0` | G0/1 | 10.0.2.1 | 255.255.255.0 | N/A |

### CML Environment Setup

1. Verify that your hosts and router `<initials>-R0` are running and have the correct interfaces and IP settings. Refer to Lab 5 if you need to reconfigure `<initials>-R0`.
2. Configure **Link Condition** constraints on the link between **Router R0 and Host H1**:
   - **Bandwidth**: Limit the link to **1 Mbps** (or 1000 Kbps).
   - **Latency**: Set a one-way delay of **50 ms** (100 ms RTT).
   - *Note: These constraints will limit link capacity, ensuring TCP enters congestion avoidance and slow-start cycles during file transfers.*

### Step-by-Step Traffic Generation & Execution:

1. **Start Packet Capture**: Right-click the link connecting **Host H0 and Router R0**, select **Packet Capture**, and click **Start**.
2. **Generate 150KB File on Host H0**: Open H0's console and create a dummy text file of exactly 150KB:

   ```
   dd if=/dev/urandom of=alice.txt bs=1k count=150
   ```

3. **Start TCP Listener on Host H1**: Open H1's console and start netcat in TCP listen mode, discarding the incoming stream:

   ```
   nc -l -p 8080 > /dev/null
   ```

4. **Send the File from Host H0**: Open H0's console and pipe the file through netcat to H1:

   ```
   nc 10.0.2.10 8080 < alice.txt
   ```

5. **Wait and Terminate**: Once the file transmission completes (the console prompt will return on H0), verify that the connection has completed. Press `Ctrl+C` on H0 to close netcat if it remains open.
6. **Download Capture**: Stop the CML packet capture and download the `.pcap` capture file to your local computer.

---

# 7. Wireshark Analysis and Questions

Open your downloaded packet capture in Wireshark, filter the packets using the `tcp` filter, and answer the following questions:

### Part A: TCP Basics and Handshake

1. **Client & Server Details**: What is the IP address and TCP port number used by the client computer ( `<initials>-H0` ) to transfer the file? What is the IP address and TCP port number used by `<initials>-H1` ?
2. **SYN Segment**: Locate the TCP SYN segment that initiated the connection.
   - What is the raw Sequence Number of this segment? (Note: To see the raw value, right-click on the sequence number in the packet details pane -> `Protocol Preferences` -> uncheck `Relative Sequence Numbers` ).
   - What flags are set in the TCP header to identify this as a SYN packet?
   - Does this segment advertise support for Selective Acknowledgments (SACK) in its Options?
3. **SYN-ACK Segment**: Locate the SYN-ACK segment sent by the server.
   - What is its Sequence Number?
   - What is the value of the Acknowledgment number field? How did the server determine this value?
   - What flags are set in the TCP header?
4. **Connection Teardown**: Locate the connection closing packets (FIN/ACK sequence).
   - What is the packet number of the first segment that initiated the teardown (FIN)? Which host sent it?
   - Outline the packet sequence exchanged to close the connection.

### Part B: Reliable Data Transfer

5. **First Data Segment**: Locate the first TCP segment containing the HTTP/netcat payload sent from H0 to H1.
   - What is the Sequence Number of this segment?
   - How many bytes of data are contained in the TCP payload (data) field of this segment?
6. **Data Acknowledgment**: Locate the ACK segment sent by the server acknowledging this first data segment.
   - What is the Sequence Number of this ACK segment?
   - What is the Acknowledgment number value? How did the server determine this value?
7. **Retransmission Check**: Are there any retransmitted segments in the trace file? What did you check in Wireshark (filters or packet fields) to answer this?

### Part C: Flow Control and Throughput

8. **Segment Lengths and Window Size**: Examine the first four data-carrying TCP segments sent from H0 to H1.
   - What is the length (header plus payload) of each of these segments?
   - What is the minimum amount of available buffer space advertised to the client by `<initials>-H1` among these first four segments? (Inspect the `Window Size Value` field).
   - Did the lack of receiver buffer space ever throttle the sender during these segments?
9. **Delayed ACKs**: How much data does the receiver typically acknowledge in a single ACK among the first ten data-carrying segments? Can you identify cases where the receiver is ACKing every other received segment (delayed ACKs)?
10. **Throughput Calculation**: What is the throughput (bytes transferred per unit time) for the TCP connection? Explain how you calculated this value (indicate the start/end timestamps and total payload bytes).

### Part D: Congestion Control in Action

11. **Stevens Graph Plot**: Click on a client-sent TCP segment in the Wireshark packet list. Then select: `Statistics -> TCP Stream Graph -> Time Sequence Graph (Stevens)` .

    > ***IMPORTANT - Deliverable Screenshot****: Take a screenshot of the generated Stevens graph. The X-axis represents time, and the Y-axis represents TCP sequence numbers. Include this graph in your report.*

12. **Congestion Phases Analysis**: Analyze the Stevens graph:
    - Describe the pattern of the dots (e.g., clusters/fleets of packets sent back-to-back, followed by flat periods).

- Can you identify where the connection is in the **slow start** phase and where it transitions to the **congestion avoidance** phase? Explain what indicators on the graph support your analysis.

13. **RTT Graphing (Optional / Extra Credit)**: Plot the RTT for the TCP segments sent from the client. Select: `Statistics -> TCP Stream Graph -> Round Trip Time Graph`.
    - What is the average RTT observed? Does this match the Link Condition latency of 100ms round-trip time?

---

# 8. Troubleshooting

**Issue Encountered:** [Describe what went wrong, if anything] **Discovery Method:** [How did you find the issue?] **Resolution:** [How did you fix it?]

*(If no issues were encountered, explain potential pitfalls for this configuration).*

---

# 9. Conclusion & Analysis

Summarize your findings regarding TCP's connection management, reliability mechanisms, and how the sliding window flow control and congestion avoidance phases ensure network stability.

---

# 10. What to Hand In

- The Lab YAML configuration file with your name, date, and name of the lab at the top in comments.
- A single PDF report containing:
  - Your written answers to the 13 Wireshark analysis questions in Section 7.
  - A screenshot of your active CML topology showing the running hosts ( `<initials>-H0` , `<initials>-H1` ) and router ( `<initials>-R0` ).
  - An annotated screenshot of your Wireshark packet list showing the TCP 3-way handshake (SYN, SYN-ACK, ACK).
  - An annotated screenshot of your Wireshark packet list showing the connection teardown (FIN, ACK, FIN, ACK).
  - The required **Time Sequence Graph (Stevens)** screenshot generated in Wireshark as detailed in Part D.